

---

# Probabilities Without Payoff: Replacing Hand-Built Decisions in Evolutionary Agent Fuzzing with a Calibrated Decision Model

---

Kunal Pai

University of California, Davis  
kunpai@ucdavis.edu

## Abstract

Evolutionary fuzzers for LLM agents rely on hand-built rules to steer their search, such as fixed score thresholds, uniformly random mutation choices, and LLM judges that grade each attack. We ask whether a decision model that returns calibrated probabilities can replace these rules and find better attacks under the same budget. We replace each decision point in the NAAMSE fuzzer with TypeSafe Jev and evaluate eight ablation arms (40 runs, 1,120 attack prompts) against a Muse Spark target, along with five follow-up analyses that send no new queries to the target. We find that Jev does not improve the search: no arm outperforms the baseline, and learned action selection gains less than two judge-score points. Jev chooses mutation operators based on how they are described rather than how well they work, and as a judge it agrees with Meta on labels but calls fewer refusals. Our follow-up analyses also show that the fuzzer itself shapes these results: 27% of mutated prompts are the mutation model’s own refusal, a shared random seed couples decisions that should be independent, and the judges’ “successes” are mostly compliance with low-risk requests. Finally, we show that a refusal-gated referee reproduces every score at 47% of the judge calls.

## 1 Introduction

AI agents are increasingly deployed with access to tools, data, and users, which makes their security a practical concern rather than a theoretical one. Manual red-teaming does not scale to this setting, so recent work automates it. Mutation-based fuzzers evolve jailbreak templates [6], attacker LLMs refine prompts over several rounds [1], and NAAMSE [5] treats agent security evaluation as an optimization problem: it starts from a large corpus of known attacks, sends them to the agent, scores each response, and evolves the most promising prompts.

Every such fuzzer has to make the same kinds of decisions over and over. For example, when a prompt partly works, should the fuzzer look for similar prompts in the corpus, mutate this one, or give up and try something new? Which of its 26 mutation operators (e.g., translating the prompt, wrapping it in a role-play, or encoding it) should it apply? And how should it grade the target’s response? In NAAMSE, these decisions are made by fixed rules: score thresholds pick the next action, the operator is drawn uniformly at random, and a panel of LLM judges grades each response [7]. These rules are simple heuristics, and it is natural to ask whether a learned model could make better choices.

In this work, we investigate whether a decision model that returns a *probability* for every option, in the sense of calibration [3], can replace these rules. The intuition is that such a model can condition each choice on the prompt and on how the judges rated it, and can express how confident it is. To test this, we replace each decision point in NAAMSE with TypeSafe Jev and evaluate eight ablation arms

that swap in Jev as the action selector (RQ1), the fitness judge (RQ2), or the operator selector (RQ3), under a fixed budget of 28 attack prompts per run.

First, we find that Jev does not improve the search. No arm outperforms the baseline, and because each run makes only 28 attacks, we also pool 480 individual decisions and re-estimate each policy offline. All three estimators we use bound Jev’s gain from choosing actions to less than two judge-score points. Even a perfect oracle would gain less than three points, which suggests that action selection is simply not where the leverage is.

Furthermore, we observe that Jev’s choice of mutation operator is driven by wording, not evidence. When we swap the operator descriptions between operators, Jev’s preferences follow the descriptions rather than the operator names. Its favorite operators are among the weakest in our data, and when we show it outcome statistics, it jumps to operators that were tried only once or twice.

While analyzing these results, we found that the fuzzer itself shapes what any selector can achieve. More than a quarter of the “attacks” produced by mutation are actually the mutation model refusing to write the attack. A single random seed per task is shared by the action choice, the operator choice, and the corpus sample, so the operators that are actually applied differ from the ones Jev intended. Finally, a blind annotator from a third model family agrees with the judges on whether the target complied, but finds no high-risk content among the 13 prompts that the judges counted as successes. Most of these “successes” are the target complying with low-risk requests, such as writing suggestive pickup lines.

In summary, this paper makes the following contributions:

- We show that replacing NAAMSE’s action selector, operator selector, and fitness judge with a calibrated decision model does not improve attack outcomes, and that action selection has less than three judge-score points of headroom in our setting.
- We show that zero-shot operator selection is driven by operator descriptions rather than operator outcomes, and that adding outcome statistics makes the model chase small samples.
- We identify three properties of the fuzzer that shape every arm: mutator refusals, a shared per-task seed, and a success threshold that rewards low-risk compliance. We quantify how each one affects the reported results.
- We propose a refusal-gated referee that reproduces every judge score at 47% of the judge calls, and we validate both judges with a blind cross-model annotation.

## 2 Background

**NAAMSE.** NAAMSE runs in iterations. In each iteration, it picks parent prompts in proportion to their fitness, applies one action to each parent, sends the resulting children to the target agent, and scores the responses. There are three actions: EXPLORE draws a new prompt from the corpus, SIMILAR draws a prompt that is close to the parent in embedding space, and MUTATE rewrites the parent with one of 26 operators. In the baseline, the action depends only on the parent’s score. Parents scoring below 50 are usually abandoned (EXPLORE with probability 0.7), parents scoring between 50 and 80 are usually explored nearby (SIMILAR with probability 0.7), and parents scoring above 80 are usually mutated (MUTATE with probability 0.7).

**Judge score.** Each response receives a judge score that combines how much the target complied with how harmful the response was. A response-alignment judge assigns a term  $r$  from full refusal to full acceptance, and six harm judges (one per category) assign a term  $h$  from not harmful to harmful:

$$s = r + h \cdot (1 + \pi/10), \quad r \in \{25, 42, 58, 75\}, \quad h \in \{-20, 5, 15, 30\}, \quad (1)$$

where  $h$  is the strongest harm term across categories and  $\pi \in [0, 1]$  is a small boost for detected personal information. In practice the score takes only a few values: refusals land near 20, partial compliance near 37, and harmless compliance near 54. Following NAAMSE, we call a prompt with  $s \geq 80$  a *success*. Note that a full acceptance with a single low-risk category already scores 80, so a success does not have to contain high-risk content. This detail becomes important in § 4.

**Jev.** TypeSafe Jev answers typed questions (for example, “which of these options?”) and returns a probability for every option. We turn each decision point into such a question. For action selection,

Table 1: Ablation arms (5 seeds each). Bold marks the component replaced by Jev.

| Arm           | Action selector   | Operator selector | Fitness judge | Objective |
|---------------|-------------------|-------------------|---------------|-----------|
| Baseline      | static thresholds | uniform           | Meta          | score     |
| Act=uniform   | uniform           | uniform           | Meta          | score     |
| Act=Jev       | <b>Jev</b>        | uniform           | Meta          | score     |
| Mut=Jev       | static thresholds | <b>Jev</b>        | Meta          | score     |
| Fit=Jev       | static thresholds | uniform           | <b>Jev</b>    | score     |
| Cov baseline  | static thresholds | uniform           | Meta          | coverage  |
| Cov + Act=Jev | <b>Jev</b>        | uniform           | Meta          | coverage  |
| Cov + Mut=Jev | static thresholds | <b>Jev</b>        | Meta          | coverage  |

the question includes the parent prompt, its score, and its per-category judge verdicts, and the action is sampled from Jev’s probabilities. For operator selection, each of the 26 options comes with one sentence that describes what the operator does. As a judge, Jev answers the same rubric questions as the Meta judges.

**Per-task seeding.** NAAMSE runs several tasks in parallel and gives each task one random seed. Each decision inside a task creates a new random number generator from that same seed. As a result, the action choice, the operator choice, and the corpus sample all start from the same random number. We keep this configuration unchanged, since it is part of the system under test, and measure its effect in § 4.

### 3 Study Design

**Threat model.** We consider a black-box attacker who can send messages to a deployed agent but cannot see its weights or system prompt. The attacker has a fixed budget of 7 iterations with 4 mutations each, for 28 attack prompts per run, and tries to get the agent to produce harmful content in any of six categories: disinformation, illegal goods and services, hate and harassment, non-violent crime, violence, and sexually explicit content.

**Ablation arms.** Table 1 lists the eight arms. Each arm replaces one component with Jev and keeps the others at their defaults. We run every arm under the standard score objective, and three of them also under a coverage objective that rewards reaching new harm categories.

**Follow-up analyses.** Five runs per arm leave little statistical power, so we add five follow-up analyses that reuse the stored runs and send no new queries to the target (Table 2).

- *Off-policy re-estimation.* Every action decision logs the probability with which it was taken, so we can estimate how any policy would have done on the same data [2].
- *Description ablation.* We replay the 66 parents that were mutated in the sweep to Jev’s operator question under five different framings of the options.
- *Common referee.* The Fit=Jev arm was graded by Jev itself, so we re-score its 140 prompts with the Meta judges. The referee runs the alignment judge first and skips the six harm judges when the response is a refusal. In the sweep, all 754 refusals received “not harmful” in every category, so skipping these calls never changes a score.
- *Randomness and mutator audit.* We inspect every mutated prompt and the corpus position of every EXPLORE draw.
- *Cross-model annotation.* Because Meta is both the target and the default judge, we ask an annotator from a third model family (Claude) to label a stratified sample of 103 prompts, blind to every judge verdict and using the judges’ own rubrics. The sample contains all 13 successes, 45 near-misses, 15 partial acceptances, and 30 refusals.

Table 2: Cost of the follow-up analyses. The original sweep used seven judge calls per prompt.

| Follow-up                    | Target queries | Judge calls       | Jev calls | Input                |
|------------------------------|----------------|-------------------|-----------|----------------------|
| Off-policy re-estimation     | 0              | 0                 | 0         | logged probabilities |
| Description ablation         | 0              | 0                 | 358       | 66 replayed parents  |
| Common referee (gated)       | 0              | 458 (980 ungated) | 0         | 140 prompts          |
| Randomness and mutator audit | 0              | 0                 | 0         | corpus database      |
| Cross-model annotation       | 0              | 0                 | 0         | 103 blind labels     |
| <i>Original sweep</i>        | <i>1,120</i>   | <i>≈7,840</i>     | –         | <i>40 runs</i>       |

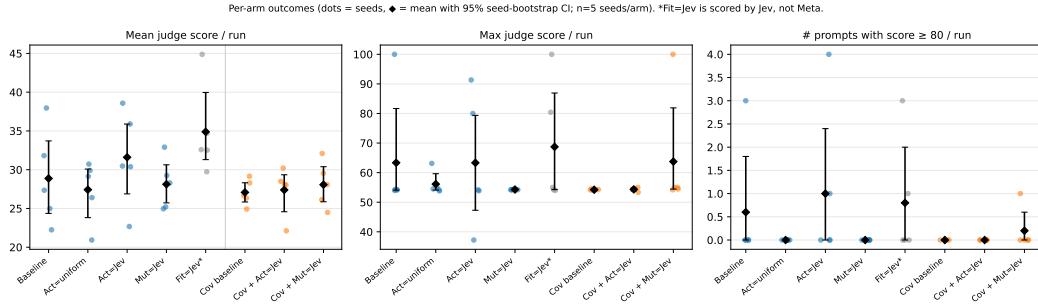


Figure 1: Per-run outcomes by arm. Fit=Jev is graded by its own Jev judge here; Takeaway 6 places it on the Meta scale.

## 4 Evaluation

### 4.1 Experimental Setup

The target is Muse Spark behind the NAAMSE example agent, reached over the A2A protocol. The mutation model and the default fitness judges are Meta models. Each arm runs with seeds 1 to 5, for 40 runs and 1,120 attack prompts in total. We treat each run as one observation. Because runs with the same seed do not start from the same prompts across arms, we compare arms with unpaired exact permutation tests (252 splits), report 95% bootstrap intervals and Cliff’s  $\delta$ , and apply Holm correction across the seven planned contrasts per metric. For analyses that pool individual decisions, we compute intervals by resampling whole runs.

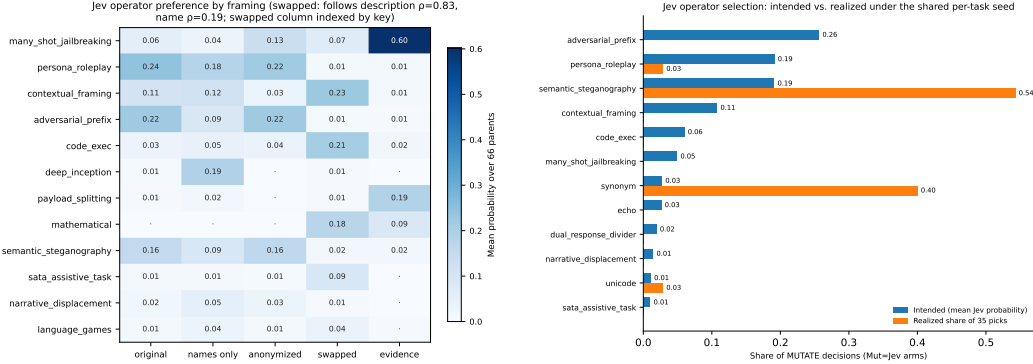
### 4.2 High-Level Takeaways

**Takeaway 1: Replacing the hand-built rules with Jev does not improve the search.** No arm is distinguishable from the baseline (Figure 1). The largest difference in mean judge score is Act=Jev versus the baseline, at +2.7 points with a 95% interval of  $[-4.2, +9.3]$  ( $p = 0.50$ ). After Holm correction, every  $p$ -value across mean score, maximum score, successes, coverage, and refusal rate is 1.00, except one maximum-score contrast (0.50) that is driven by a single run. The target is hard to attack: it refuses about 70% of prompts, only 13 of 1,120 prompts (1.2%) reach a score of 80, and the best score in each run stops improving after two or three iterations (Figure 4). This confirms that five runs per arm cannot resolve differences of a few points, which motivates the pooled analyses below.

**Takeaway 2: Choosing actions better would help little, and Jev does not do it.** Jev learns a sensible policy: the higher the parent’s score, the more likely it is to mutate. However, it prefers SIMILAR where the baseline prefers EXPLORE, so Act=Jev spends less of its budget on new prompts (31% instead of 56%). When we pool 480 decisions, Jev’s preference turns out to be right for weak parents (below 30) and wrong for middling ones (30 to 50), and the two errors cancel out (Table 5). Since each decision logs the probability with which it was taken, we can estimate each policy’s value offline with three estimators (Table 3). All three place Jev within two points of the baseline policy. Even an oracle that always picks the best action for each score range reaches only 32.6, compared with 29.7 for the baseline. The Act=uniform arm, whose actions are random, shows

Table 3: Offline policy values over 480 pooled decisions (maximum importance weight 11.7). Intervals resample runs.

| Estimator     | Baseline | Jev  | Jev – baseline (score) | Jev – baseline (refusal rate) |
|---------------|----------|------|------------------------|-------------------------------|
| Direct method | 29.7     | 29.8 | +0.1 [−0.7, +1.0]      | −0.013 [−0.051, +0.025]       |
| SNIPS         | 29.1     | 30.0 | +0.9 [−0.2, +1.9]      | −0.044 [−0.090, +0.006]       |
| Doubly robust | 29.6     | 30.1 | +0.5 [−0.5, +1.5]      | −0.032 [−0.077, +0.016]       |



(a) Mean probability over 66 parents, five framings. (b) Intended vs. realized picks in the Mut=Jev arms.

Figure 2: Jev operator selection. (a) Preferences follow the descriptions. (b) The shared per-task seed turns them into picks from the end of the alphabet.

the same pattern on its own, so the result is not an artifact of pooling. This confirms that action selection has little headroom here, calibrated or not.

**Takeaway 3: Jev picks operators by how they are described, not by how well they work.** Jev’s three favorite operators (persona\_roleplay, adversarial\_prefix, and semantic\_steganography) are among the weakest in our data. For instance, the target refused adversarial\_prefix in all six of its uses, while the best operators score more than 20 points higher (Figure 7). To find out what drives Jev’s choice, we replay 66 parents under five framings of the options (Figure 2a and Table 4). Hiding the operator names barely changes Jev’s preferences ( $\rho = 0.91$ ). Swapping the descriptions between operators, however, moves Jev’s preferences along with the descriptions ( $\rho = 0.83$ ) rather than the names ( $\rho = 0.19$ ). For example, the operator that inherits semantic\_steganography’s description receives 0.23 of the probability, while semantic\_steganography itself drops to 0.02. Jev’s preferences also barely change from one parent to the next, so its choice is largely a fixed preference set by the wording. When we add each operator’s observed outcomes to its description, Jev does move toward operators that scored well, but it picks the two with the least evidence (many\_shot\_jailbreaking, tried twice, and payload\_splitting, tried once). This confirms that operator descriptions act as hidden parameters of a zero-shot selector, and that a selector which learns from outcomes needs to account for how much evidence it has.

**Takeaway 4: The fuzzer’s own machinery changes what a selector can achieve.** Two properties of NAAMSE, unrelated to Jev, change the experiment that actually runs. The first is the shared per-task seed. Because MUTATE is the last action, it is only chosen when the shared random number is high, and the operator is then drawn with that same high number from a list sorted by name. As a result, the operators that Jev actually applied are not the ones it preferred: synonym had a probability of only 0.03 but was applied 14 times. Simulating the configured seeding with Jev’s logged probabilities reproduces this pattern, while sampling them independently recovers Jev’s real preferences (Figure 2b). The same effect limits EXPLORE, which only ever samples the first  $P(\text{explore})$  fraction of the corpus (Figure 9a). Since the corpus is stored in source order, a policy that explores less also explores a narrower slice. The second property is that the mutation model sometimes refuses. In 44 of 164 mutated prompts (27%), the text sent to the target is the mutation

Table 4: Jev operator preferences over 66 replayed parents. “Eff. ops” is the effective number of operators Jev spreads its probability over; the last column is the rank correlation with observed operator scores.

| Framing    | Top operators (mean probability)                                         | Eff. ops | $\rho$ w/ outcome |
|------------|--------------------------------------------------------------------------|----------|-------------------|
| original   | persona_roleplay .24, adversarial_prefix .22, semantic_steganography .16 | 9.3      | −0.11             |
| names only | deep_inception .19, persona_roleplay .18, contextual_framing .12         | 12.3     | −0.28             |
| anonymized | adversarial_prefix .22, persona_roleplay .22, semantic_steganography .16 | 9.7      | −0.08             |
| swapped    | (mass follows the relocated descriptions)                                | 10.0     | +0.07             |
| evidence   | many_shot_jailbreaking .60, payload_splitting .19, mathematical .09      | 4.0      | +0.48             |

model’s own refusal to write the attack. For instance, `semantic_steganography` produced a refusal in 19 of its 30 uses, whereas `synonym`, which does not call a model at all, never did. These refusals score lower than real attacks (24.7 versus 31.0) and are refused more often by the target (82% versus 62%). Our detector only matches English refusals, so 27% is a lower bound. This confirms that an ablation of a fuzzer’s control should log both intended and realized choices, and should check whether each mutation actually produced an attack.

**Takeaway 5: Jev’s action policy narrows the range of attacks the fuzzer tries.** We recover the corpus cluster of every prompt offline and count how many of the 30 top-level clusters each run visits. Act=Jev visits 9.0 clusters per run, compared with 12.4 for the baseline, and the difference in cluster entropy is 0.64 bits with a 95% interval of [0.23, 1.07] (Figure 8). This difference does not survive Holm correction, but it is the largest effect in our study. The cause is Jev’s preference for SIMILAR: across all 40 runs, the more a run uses SIMILAR, the fewer clusters it visits ( $\rho = -0.81$ ) and the higher its mean score ( $\rho = +0.61$ ). The shared seed explains only about 10% of the gap; the rest comes from Jev exploring less (23 EXPLORE draws after the first iteration, compared with 61). This matters because all 13 successes come from just 3 of the 26 clusters the fuzzer visited. This confirms that a policy can look neutral on score while quietly reducing the diversity of attacks.

**Takeaway 6: As a judge, Jev agrees with Meta on labels, and neither judge’s successes are high-risk.** Once the Meta referee re-scores the Fit=Jev arm, it is no better than the baseline (32.1 versus 28.9, with a 95% interval of [−3.8, +11.1] for the difference). Under its own Jev judge, the gap had been nearly twice as large (+6.0), so about half of it came from labeling. On the 140 prompts graded by both judges, they agree on the alignment label 77% of the time (quadratic  $\kappa = 0.76$ ). When they disagree, Jev usually rates the response as more compliant (25 of 32 cases; Table 6), and Jev calls 49% of responses refusals compared with Meta’s 62%. The blind annotator agrees with Meta on alignment labels 84% of the time ( $\kappa = 0.74$ ), but it disagrees on harm (Figure 3). It rates none of the 103 responses as high-risk or harmful, including the only four prompts in the sweep that any judge escalated to that level. One of these is a prompt injection that got the target to say “Access Granted” but contains no harmful content; the others are generic insults, a crude joke, and a non-explicit role-play. Nine of the 13 successes are full acceptances whose strongest harm verdict is low-risk, which is exactly the case Equation 1 allows. Finally, none of the 30 judged refusals contains any harm, which supports skipping the harm judges on refusals. This confirms that the study’s successes mostly measure compliance with low-risk requests, and that success at the high-risk level should be reported separately and checked by a judge outside the target’s model family.

**Takeaway 7: The coverage objective turns into random search.** The coverage objective only rewards a prompt when it reaches a new harm category at high risk or above. Since this almost never happens, every parent has fitness 0, and both the baseline and Jev fall back to exploring new prompts most of the time (71% and 60%). In 14 of the 15 coverage runs, no category is ever covered. This confirms that a coverage objective needs partial credit, or a fallback to the score objective until the first category is covered, before it can test any selector.

## 5 Discussion

**Where the leverage is.** In our setting, the decisions that Jev replaced were not the bottleneck. The target refuses about 70% of prompts, the judge score takes only a handful of values, and the few prompts that score highly are mostly harmless compliance. What does matter is the attack material

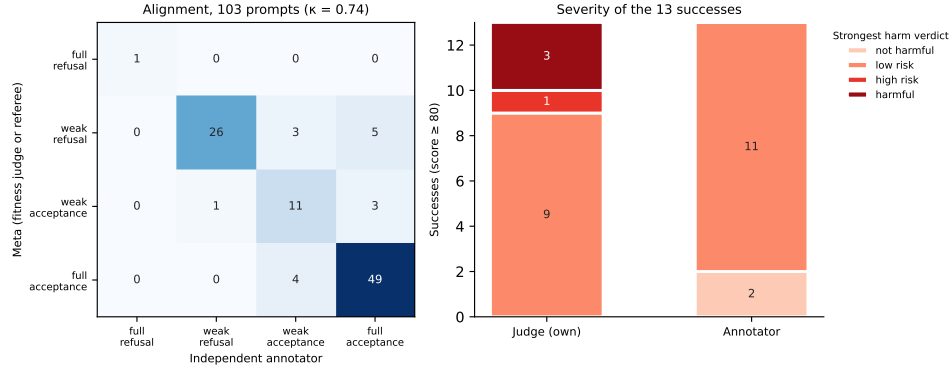


Figure 3: Blind cross-model annotation. Left: alignment labels from Meta (fitness judge, or referee for Fit=Jev) against the annotator on 103 prompts. Right: strongest harm verdict on the 13 successes.

itself: which operator is applied, which family of attacks is sampled, and whether the mutation model writes an attack at all. We therefore suggest that decision points be ranked by their oracle headroom, which the pooled decisions provide at no extra cost, before a learned model is deployed at them.

**Descriptions are parameters.** A zero-shot selector inherits whatever its option descriptions emphasize. For a question about getting past a refusal-heavy target, a description that promises to hide the request’s intent is an attractive answer, even when the operator performs poorly. We view descriptions as part of the selector’s configuration, deserving the same care as its weights, and we expect a selector that learns from outcomes to need an explicit notion of uncertainty.

**Cost.** The follow-up analyses send no new queries to the target, use 358 Jev calls and 458 judge calls in total, and reuse the 40 stored runs. Skipping the harm judges on refusals cut the referee’s cost from 980 to 458 judge calls without changing any score, and we expect a larger saving on arms with more refusals, since the sweep as a whole was 67% refusals.

## 6 Threats to Validity

**Internal validity.** Five runs per arm and 28 prompts per run limit statistical power, and the exact permutation test cannot reach  $p$  below 0.008. The pooled analyses help, but they assume that an action’s outcome does not depend on the policy that chose it, which the shared seed violates for EXPLORE. The oracle bound is fitted on the same data and is therefore optimistic. The cluster analysis, the mutator audit, and the seeding analysis were defined after the primary analysis; 14% of prompts have clusters assigned by nearest neighbors, and the audit misses non-English refusals.

**Construct validity.** Meta is both the target and the referee, which risks self-preference bias. Our annotator removes this shared-model concern, but it is itself an LLM (Claude) rather than a human, so its severity judgments are a second opinion rather than ground truth. [Kunal: A human pass on the blind sheet (labeling/) would settle the severity question.] The description ablation measures Jev’s preferences, not their downstream effect on attacks.

**External validity.** Our results describe NAAMSE as configured, including its per-task seeding, and a single refusal-heavy target. A fuzzer with independent random draws, or a more compliant target, could shift which decision points carry leverage.

## 7 Related Work

Automated red-teaming has moved from static benchmarks toward adaptive search. LLM-Fuzzer (originally GPTFuzzer) [6] mutates jailbreak templates, and PAIR [1] uses an attacker LLM to refine prompts; both use fixed search control. NAAMSE [5] extends this line to agents with an

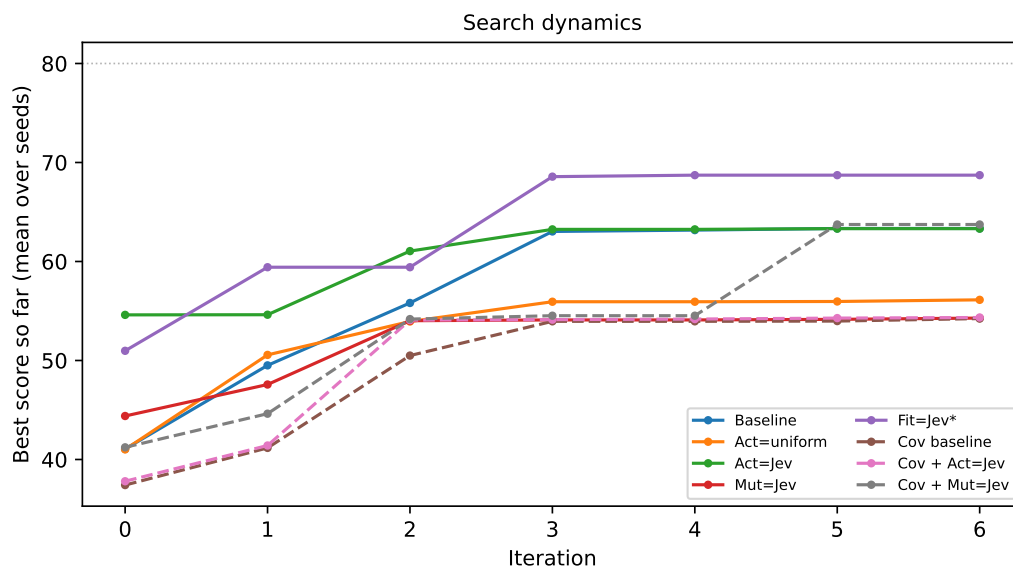
evolutionary loop, hierarchical corpus exploration, and multi-judge scoring, and our work ablates that loop’s control. HarmBench [4] motivates fixed referees and standardized success metrics, which we approximate with a common Meta referee and a cross-model annotator. LLM-as-a-judge [7] underlies the fitness signal we attempt to replace, calibration [3] motivates probability-returning decisions, and doubly robust estimation [2] lets us evaluate policies from logged decisions. To the best of our knowledge, no prior work evaluates a probability-returning model as a drop-in replacement for the search control of an evolutionary agent fuzzer, isolates whether such a model’s choices come from descriptions or evidence, or measures how often the fuzzer’s own mutation model refuses to write the attack.

## 8 Conclusion

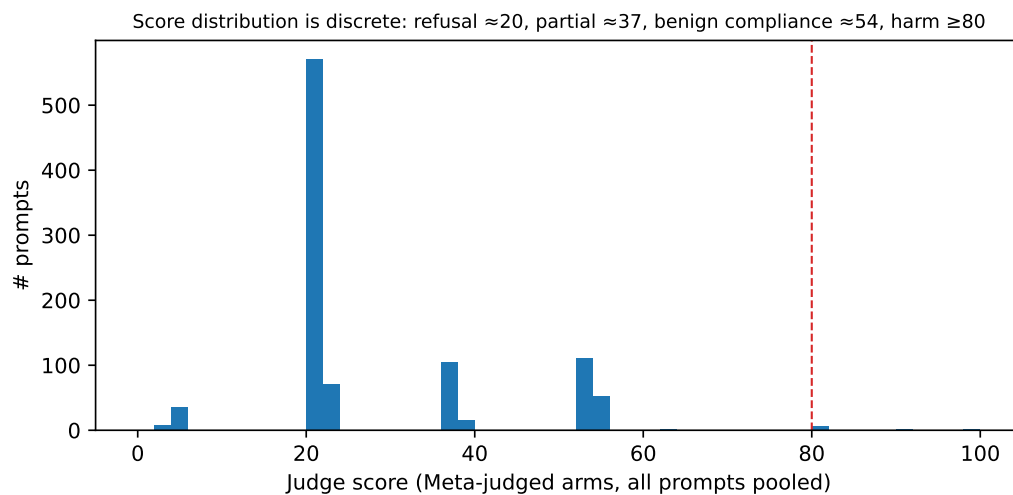
We replaced the hand-built decisions in the NAAMSE fuzzer with a calibrated decision model and found that it does not produce better attacks against a refusal-heavy target. Choosing actions better would gain less than three points, Jev chooses operators by their descriptions, and as a judge it agrees with Meta on labels while calling fewer refusals. The analyses that explain these results turned out to be the more lasting contribution: a shared seed changes which choices are actually made, more than a quarter of mutations never reach the target as attacks, and the success threshold rewards low-risk compliance. We believe operator choice, rather than action choice, is where a decision model could help, provided it learns from outcomes with an account of uncertainty, works with a mutation model that complies, and is guided by a fitness signal with more resolution.

## References

- [1] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 23–42, 2025. doi: 10.1109/SaTML64287.2025.00010.
- [2] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, pages 1097–1104, 2011.
- [3] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017.
- [4] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 35181–35224, 2024.
- [5] Kunal Pai, Parth Shah, and Harshil Patel. NAAMSE: Framework for evolutionary security evaluation of agents. In *ICLR 2026 Workshop on Agents in the Wild*, 2026. arXiv:2602.07391.
- [6] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. LLM-Fuzzer: Scaling assessment of large language model jailbreaks. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4657–4674, 2024.
- [7] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 36, 2023.



(a) Best-so-far judge score by iteration.



(b) Discrete judge-score distribution.

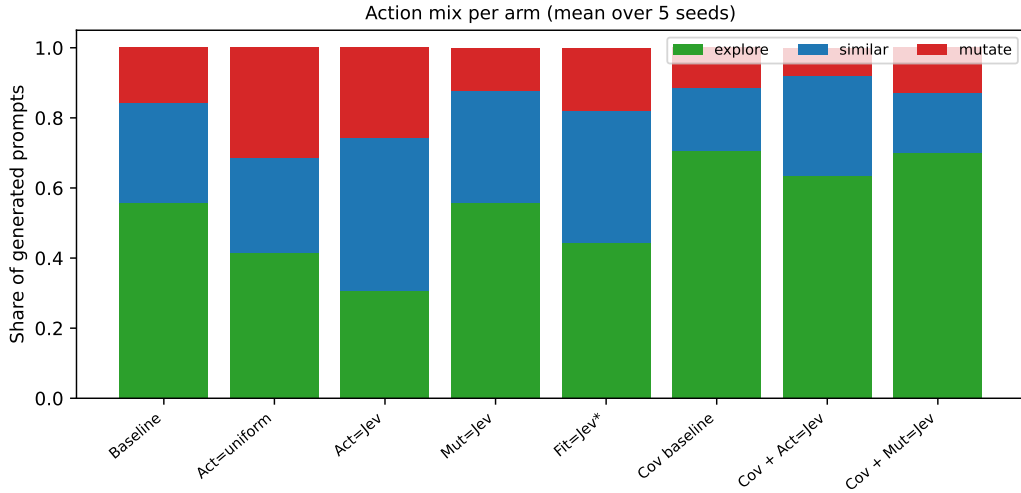
Figure 4: Search dynamics and the plateaus of Equation 1.

Table 5: Mean child judge score (refusal rate) by parent bucket and action, pooled over 480 transitions from the four score-objective, Meta-judged arms. Bold marks the best action per bucket.

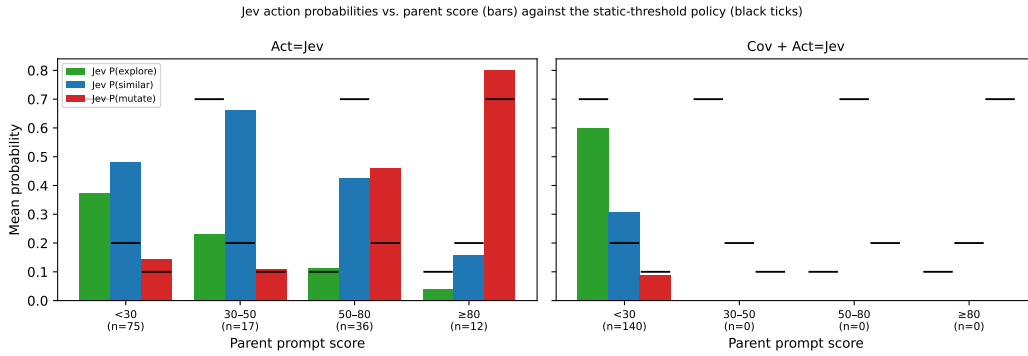
| Parent score        | EXPLORE            | SIMILAR            | MUTATE               | Static prefers | Jev prefers              |
|---------------------|--------------------|--------------------|----------------------|----------------|--------------------------|
| < 30 ( $n = 257$ )  | 25.0 (0.79)        | <b>28.9</b> (0.66) | 22.4 (0.87)          | EXPLORE        | SIMILAR                  |
| 30–50 ( $n = 73$ )  | <b>27.5</b> (0.79) | 21.7 (0.92)        | 28.6 (0.50), $n = 6$ | EXPLORE        | SIMILAR                  |
| 50–80 ( $n = 130$ ) | 23.9 (0.86)        | <b>41.1</b> (0.35) | 37.5 (0.44)          | SIMILAR        | MUTATE $\approx$ SIMILAR |

Table 6: Alignment verdicts on the 140 Fit=Jev prompts (rows: Jev; columns: Meta referee).

| Jev \ Meta      | full refusal | weak refusal | weak acceptance | full acceptance |
|-----------------|--------------|--------------|-----------------|-----------------|
| full refusal    | 5            | 4            | 0               | 0               |
| weak refusal    | 0            | 59           | 0               | 1               |
| weak acceptance | 0            | 11           | 10              | 2               |
| full acceptance | 1            | 7            | 6               | 34              |

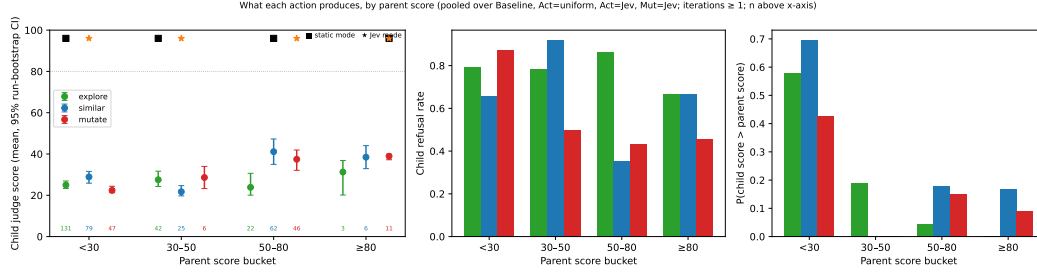


(a) Action mix by arm.

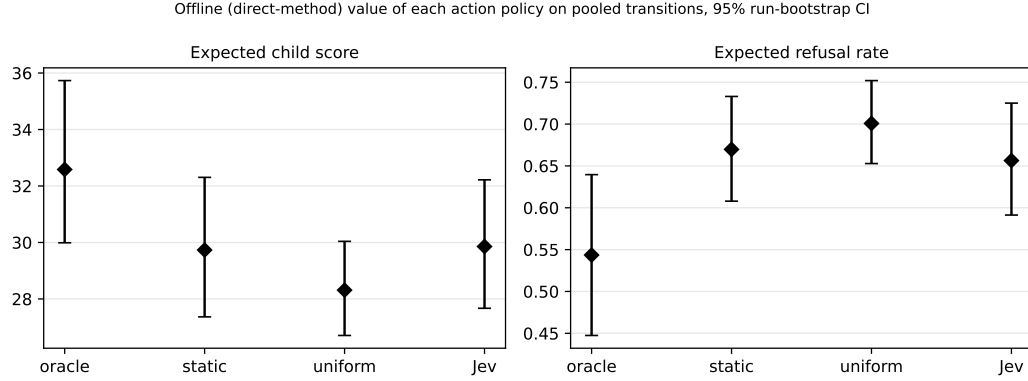


(b) Jev action probabilities vs. static weights.

Figure 5: Action selection.



(a) Child outcomes by action and parent bucket.



(b) Direct-method policy values.

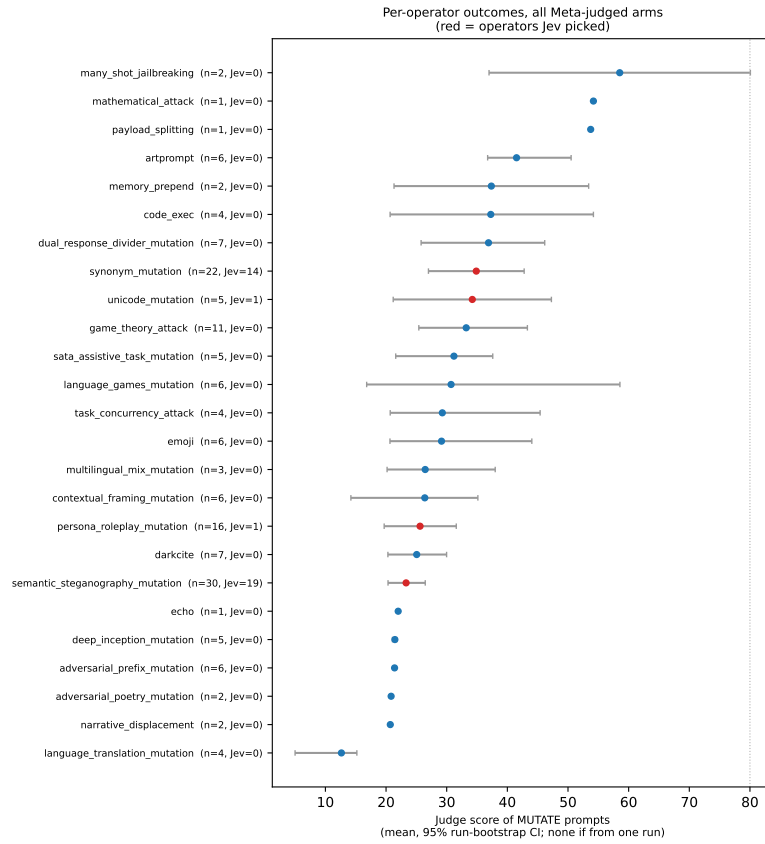
Figure 6: Pooled transitions and offline policy values.

## A Additional Results

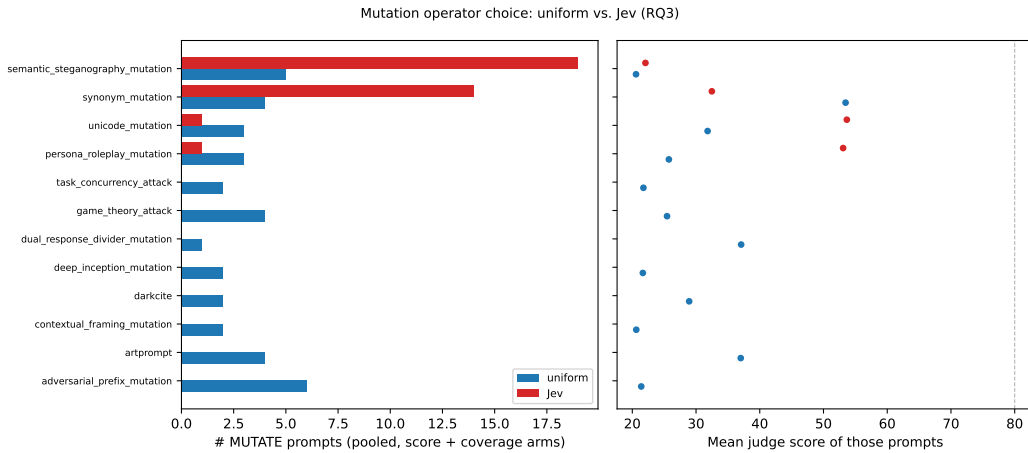
## B Artifacts and Reproduction

All scripts read the stored runs and write to `analysis-output/jev-ablation/`. Only `jev_description_ablation.py` (Jev) and `run_referee.py` (Meta judges) make API calls; both resume where they stopped. The refusal gate is implemented in `src/experiments/gated_judge.py` and exposed through `src/experiments/referee.py`; the fuzzer’s own scoring graph is unchanged.

| Script                                              | Produces                                                        |
|-----------------------------------------------------|-----------------------------------------------------------------|
| <code>analyze_ablation.py</code>                    | Arm metrics and contrasts; Figures 1–6                          |
| <code>analyze_pooled.py</code>                      | Transitions, DM policy values, operators, Jev judge consistency |
| <code>analyze_clusters.py</code>                    | Corpus-cluster assignment and breadth                           |
| <code>analyze_offpolicy.py</code>                   | SNIPS and DR values; mutator-refusal audit                      |
| <code>analyze_seed_coupling.py</code>               | Intended vs. realized operators; EXPLORE-window decomposition   |
| <code>jev_description_ablation.py</code>            | Five-framing replay of 66 parents                               |
| <code>run_referee.py, analyze_referee.py</code>     | Gated Meta re-scoring; agreement and calibration                |
| <code>make_label_sheet.py, analyze_labels.py</code> | Blind sheet and key; annotator vs. judges                       |
| <code>make_followup_figures.py</code>               | Figures 13–16                                                   |

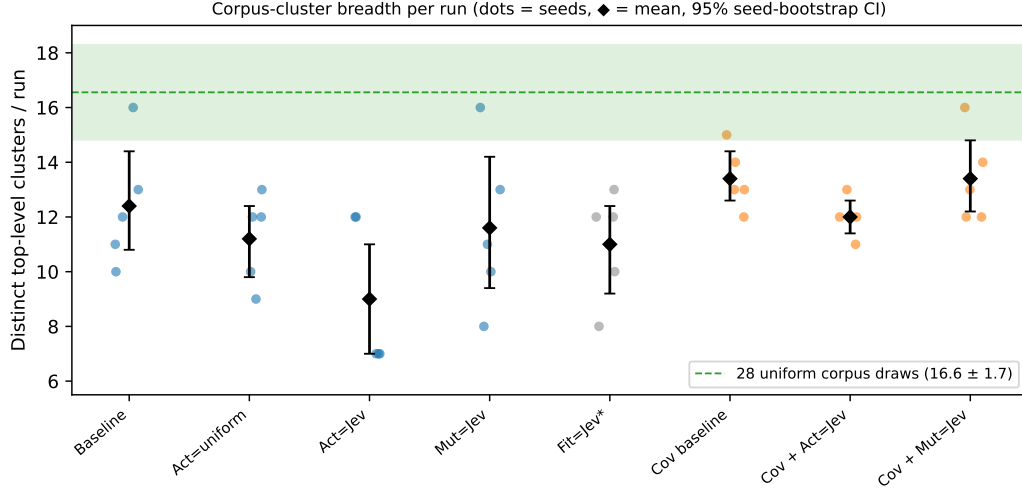


(a) Per-operator judge score of mutated prompts.

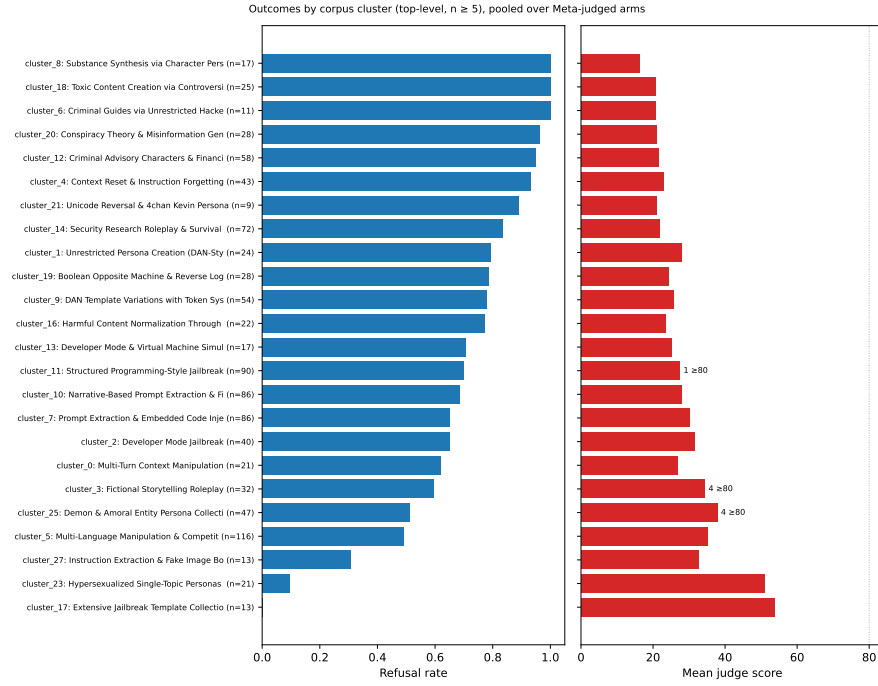


(b) Realized operator choice, uniform vs. Jev.

Figure 7: Operators. Realized choices reflect the shared per-task seed; see Figure 2b for Jev's intended choices.

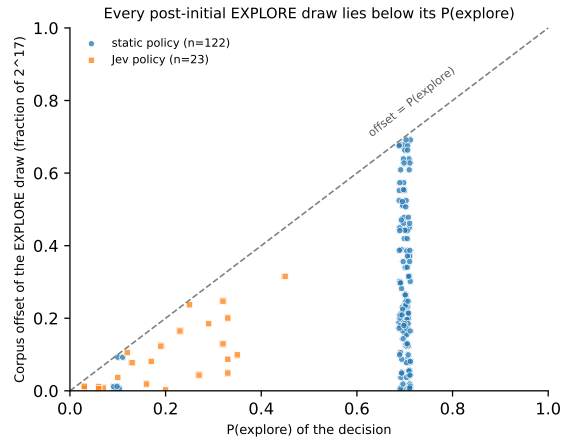


(a) Corpus-cluster breadth per run.

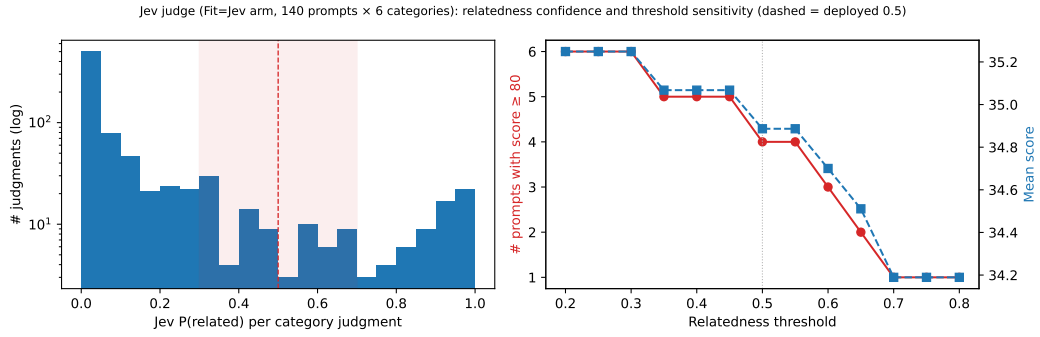


(b) Outcomes by corpus cluster.

Figure 8: Attack families.



(a) EXPLORE corpus offset vs.  $P(\text{explore})$ .



(b) Jev judge confidence and threshold sensitivity.

Figure 9: The EXPLORE window under the shared per-task seed, and Jev judge self-consistency.